The Journal of Open Source Software

couplr: Optimal pairing and matching via linear assignment

Gilles Colling [1¶]

1 University of Vienna, Austria ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`couplr` (Colling, 2026) is an R package for optimal pairing, matching, and assignment. It solves linear assignment problems: given two sets of objects and a cost for each possible pair, choose the pairs with minimum total cost. That simple operation appears in causal inference, experimental design, task allocation, sample pairing, and image alignment.

The package provides two entry points. High-level matching functions such as `match_couples()`, `ps_match()`, `full_match()`, `cem_match()`, and `subclass_match()` prepare data, build distances, enforce constraints, and return analysis-ready matched data. Low-level functions such as `lap_solve()`, `lap_solve_batch()`, `lap_solve_kbest()`, and `assignment_duals()` expose a collection of assignment solvers directly for users who already have a cost matrix. This lets the same package support both statistical matching workflows and general-purpose assignment tasks.

Statement of need

Matching is a common way to make treated and control groups more comparable before downstream analysis. In practice, researchers often need more than a single nearest-neighbor match on a propensity score. They may need exact blocking by site, calipers on multiple variables, pairwise distances on observed covariates, variable-ratio full matching, replacement matching, or direct control over the assignment algorithm. Those choices are scattered across R packages and usually require users to move between data preparation, distance calculation, solver calls, and balance checks by hand.

`couplr` packages this workflow around the linear assignment problem. Users can match directly on observed covariates, add blocking and distance constraints, inspect standardized differences and distributional diagnostics, then join the matched groups for analysis. The lower-level interface remains available when the matching problem has already been reduced to a cost matrix, which matters for operations research and scientific computing use cases where rows and columns are not treated and control units.

This split is important. A causal-inference user should not need to know which solver handles rectangular matrices well, and an operations researcher should not need the package to impose a statistical matching vocabulary. `couplr` keeps those layers separate while sharing the same tested assignment core.

State of the field

Several R packages address parts of this problem. `MatchIt` provides a widely used interface for preprocessing and matching in causal inference, with strong support for propensity score workflows and downstream balance assessment (Ho et al., 2011). `optmatch` implements

optimal and full matching through network flow formulations (Hansen & Klopfer, 2006). `cobalt` supplies balance diagnostics that work across matching packages (Greifer, 2025). For the linear assignment problem itself, R users often reach for packages such as `clue`, `lpSolve`, or domain-specific wrappers around one or two solvers (Berkelaar & others, 2024; Hornik, 2024).

`couplr` is designed for the gap between these layers. It provides a model-light interface for direct covariate matching, exports compatibility methods for `MatchIt` and `cobalt`, and includes a broad collection of native C++ solvers under a single R API. The solver set covers classical dense assignment, sparse and rectangular variants, k-best assignment, bottleneck assignment, min-cost flow formulations, and entropy-regularized transport. This lets users start with a statistical matching task and still keep access to the algorithmic machinery when the problem size or structure changes.

Software design

The package has three main layers. The first layer validates inputs and converts data frames into cost matrices. It handles scaling, missing data checks, blocking, calipers, maximum-distance rules, and user-supplied weights. The second layer solves the assignment problem. It includes implementations of the Hungarian method (Kuhn, 1955), Jonker-Volgenant shortest augmenting paths (Jonker & Volgenant, 1987), auction algorithms (Bertsekas, 1988), cost scaling (Goldberg & Kennedy, 1995), Gabow-Tarjan scaling (Gabow & Tarjan, 1989), push-relabel min-cost flow ideas (Goldberg & Tarjan, 1988), network simplex, Ramshaw-Tarjan rectangular assignment (Ramshaw & Tarjan, 2012), and related specialized solvers. The third layer returns structured result objects with pairs, costs, diagnostics, weights, subclass information, and conversion methods for other R matching tools.

The input layer supports several distance choices and constraint types. Distances are computed from numeric covariate columns using Euclidean, Manhattan, Mahalanobis, Chebyshev, or squared-Euclidean metrics, or from a user-supplied function; propensity-score distances are available through a separate entry point. Variables can be standardized to mean zero and unit variance, rescaled to the unit interval, or robust-scaled using the median and median absolute deviation. Per-variable weights are folded into the distance by multiplying each column by the square root of its weight, so that squared differences accumulate with the intended weighting. Three constraint mechanisms reshape the cost matrix before it reaches the solver: a global `max_distance` ceiling, per-variable calipers expressed as absolute-difference thresholds, and explicit forbidden pairs supplied as index pairs. Forbidden cells receive a large finite cost rather than infinity, which keeps the matrix usable across all solvers, including those that assume finite arithmetic. Blocking is built on top of these mechanisms: stratified matching either reuses an existing factor variable or constructs blocks via k-means or hierarchical clustering, and the solver is then called once per block.

The automatic solver selection in `lap_solve(method = "auto")` uses matrix shape, sparsity, cost type, and problem size to choose a method. Dense square problems can use fast dense solvers such as Jonker-Volgenant or cost scaling; sparse matrices can use sparse augmenting-path solvers; rectangular problems can avoid padding through rectangular assignment methods. Integer scaling solvers are exposed where their assumptions are met. For example, the package documents Gabow-Tarjan as an integer scaling method with theoretical dense square complexity $O(n^3 \log C)$, or $O(n^3 \log(nC))$ under the common bound that includes problem size and maximum cost. Figure 1 shows observed median solve time across problem sizes for all 18 solvers; benchmarks are reproducible via `paper/make-figure.R`.

The Journal of Open Source Software

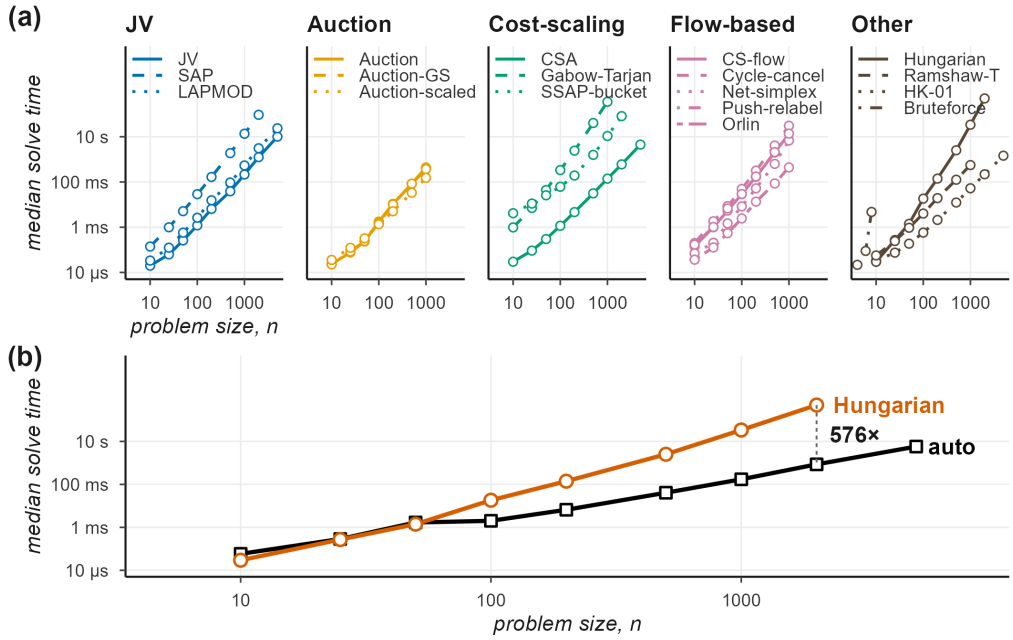

Figure 1: (a) Median wall-clock solve time versus problem size n for all 17 assignment solvers in `couplr`, arranged as five small-multiple panels grouped by algorithm family (JV / augmenting-path, Auction, Cost-scaling, Flow-based, and Other; panel headers abbreviated for space). Within each panel, individual solvers share a single family colour and are distinguished by line style; all panels share log–log axes. The *Other* panel collects single-solver families and special cases: Hungarian (classical baseline), Ramshaw–Tarjan (rectangular), HK-01 (binary $0/1$ costs, its target regime), and Bruteforce (only evaluated at $n \in \{4, 6, 8\}$). (b) The package's `auto` dispatcher (■) against the classical Hungarian baseline (●); annotated points mark the speed-up at diagnostic n. Each point is the median of 5 replicates on a single core; integer cost matrices with entries in $[1, 10{,}000]$. Reproducible from `paper/make-figure.R`.

The matching layer returns ordinary R objects with print, summary, plot, and join methods. This keeps the output usable in base R, tidyverse pipelines, and causal-inference workflows that expect matched data, weights, or subclasses.

Research impact statement

`couplr` is already released on CRAN and has continuous integration across Windows, macOS, and Linux. Its documentation includes vignettes for getting started, matching workflows, algorithm selection, troubleshooting, and pixel-level morphing examples. The package is useful for experimental designs where samples must be paired before measurement, observational studies that need transparent covariate matching, and allocation problems where each object must be assigned once under costs and constraints.

The research contribution is practical: `couplr` brings several assignment methods into one R package and connects them to preprocessing, constraints, balance diagnostics, sensitivity analysis, and joined output. The result is a shorter path from a scientific matching question to a reproducible set of pairs.

AI usage disclosure

The package was developed using a modern AI-assisted developer stack. The author designed the package architecture, made the algorithm-selection choices, and wrote the implementation,

DRAFT

using a self-hosted Qwen3-Coder-Next-REAP-48B-A3B model (48B-parameter mixture-of-experts coder, ~3B active per token, Router-weighted Expert Activation Pruning, 4-bit MLX quantization, running on-device on a single Apple M4 Pro with no cloud inference) for code completion and routine generation, integrated through Anthropic's Claude Code CLI configured to route to the local model.

Acknowledgements

No external funding supported this work.

References

Berkelaar, M., & others. (2024). *lpSolve: Interface to lp_solve v. 5.5 to solve linear/integer programs*. https://CRAN.R-project.org/package=lpSolve

Bertsekas, D. P. (1988). The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, *14*, 105–123. https://doi.org/10.1007/BF02186476

Colling, G. (2026). *Couplr: Optimal pairing and matching via linear assignment*. https://doi.org/10.32614/CRAN.package.couplr

Gabow, H. N., & Tarjan, R. E. (1989). Faster scaling algorithms for network problems. *SIAM Journal on Computing*, *18*(5), 1013–1036. https://doi.org/10.1137/0218069

Goldberg, A. V., & Kennedy, R. (1995). An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, *71*(2), 153–177. https://doi.org/10.1007/BF01585996

Goldberg, A. V., & Tarjan, R. E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, *35*(4), 921–940. https://doi.org/10.1145/48014.61051

Greifer, N. (2025). cobalt: Covariate balance tables and plots. *Journal of Open Source Software*, *10*(109), 7610. https://doi.org/10.21105/joss.07610

Hansen, B. B., & Klopfer, S. O. (2006). Optmatch: Flexible, optimal matching for observational studies. *The R Journal*, *2*(2), 29–36. https://journal.r-project.org/archive/2006-2/RJournal_2006-2_Hansen+Klopfer.pdf

Ho, D. E., Imai, K., King, G., & Stuart, E. A. (2011). MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, *42*(8), 1–28. https://doi.org/10.18637/jss.v042.i08

Hornik, K. (2024). *Clue: Cluster ensembles*. https://CRAN.R-project.org/package=clue

Jonker, R., & Volgenant, T. (1987). A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, *38*(4), 325–340. https://doi.org/10.1007/BF02278710

Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1-2), 83–97. https://doi.org/10.1002/nav.3800020109

Ramshaw, L., & Tarjan, R. E. (2012). *On minimum-cost assignments in unbalanced bipartite graphs* (HPL-2012-40R1). HP Laboratories. https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html

DRAFT